The Relay

Files:

- Code is in `relay/`.
- The raw rows are in `results/results.csv`: one row per request, 797 rows (219 real, 578 stand-in). The columns after `answer_grade` are extras: model, cache hit, degrade level, shed reason, wait/work/retry ms, and calls.
- Per-request traces, including the logged drafts, are in `results/traces.jsonl`.
- The real-service probes are in `results/probe_raw.jsonl`.
- `python -m relay.finalize && python -m relay.analyze results/results.csv` rebuilds every table.

1 What I built and why

The pipeline has five steps:

1. **Understand.** gpt-oss-20b returns JSON: whether the question is in scope, the paper, and search words.
2. **Retrieve.** A BM25 tool over 250-word pieces of the papers. It has a 1.5 s timeout and falls back to an in-process search.
3. **Decide.** Refuse without drafting if nothing relevant is found. Otherwise choose the model and passage count from the remaining budget.
4. **Draft.** gpt-oss-120b, streamed, citing passages as [n].
5. **Verify.** A deterministic check that gates what the person sees. Every claim must cite a passage it was given, and every number in it must appear in that passage. If the first sentence fails, the system redrafts once (the loop). If it fails again, it refuses and names the closest passages.

The decision that shaped it most is that the scarce resource is tokens per minute, not requests. Each model allows 8,000 TPM, and a real answer costs 2,174 tokens on average (prompt, hidden reasoning and answer), so each model produces about 3.5 answers a minute. That's why model calls only go where they change the answer:

- The checker is code, not a second judge call.
- Out-of-scope questions are refused before drafting.
- Repeated questions are served from a cache.
- A client-side governor predicts each request's wait. It turns away anyone it can't serve within 20 s straight away, rather than letting them time out.

I define "fast" as the first *verified* sentence at p50 under 2 s and p95 under 10 s when not overloaded. Gating on verification costs time to first token, because a sentence is held until it is checked. I accept that cost, because the brief rules out sending unchecked text.

2 The real service and my stand-in

I sent 165 real calls to measure the service: 25 tiny and 25 RAG-sized prompts per model, 40 calls at once, 12 back-to-back 697-token prompts, and 12 concurrent calls. The results are in Table 1.

The finding that changed the design: Groq's limits refill continuously; they don't reset on fixed windows.

- After a per-minute 429 saying "try again in 2s", a call 2.7 s later succeeded.
- In the TPM test, `x-ratelimit-reset-tokens` grew about 5.2 s per 697-token call, which is 8,000 tokens ÷ 60 s.
- The *daily* limit's reset header read 1h50m52s after 77 calls, which is 86.4 s each.
- When the daily token limit was reached, the 429 said retry-after 696 s. That's the time a ~1,600-token request takes to refill at 200,000 tokens ÷ 86,400 s.

So I rebuilt the governor as refilling buckets. My first version used one-minute windows.

The stand-in (`relay/standin.py`) is an OpenAI-compatible server that streams its reply piece by piece.

- **Calibrated from the probes:**
  - Time to first token is lognormal: median 469 ms with σ 0.61 for 120b, and 500 ms with σ 0.29 for 20b.
  - Streaming speed: 1,222 and 2,190 tokens/s.
  - 4% slow successes at 12× the median.
  - Groq-shaped 429 bodies and headers, with buckets that refill like Groq's.
- **Behaviour:** it fails 0.5% of calls before the first token and 0.3% mid-stream. It slows down past 8 calls in flight. It emits hidden "reasoning" tokens before visible text.
- **Where it is least like Groq:**
  - Its p50 is too optimistic: 730 ms in the stand-in steady run against 1,944 ms in the real steady run. Real gpt-oss reasoning delays the first token more than my 80-token assumption.
  - The slowdown under load is invented. 12 concurrent real calls showed none.
  - Its answers echo passage 1, so stand-in runs measure latency and outcomes, never quality.
  - It never cites as 【n】, which the real 120b model does (§8).

3 Results

Percentiles are nearest-rank over status=ok rows: p50, with p95/p99 as the slow end. The "Requests sent" column gives n. Only `steady-real` and the quality runs used the real service; everything else used the stand-in. Cache hits count as served answers; cache share was 15% (steady), 29% (burst) and 38% (overload).

A number I could not have guessed: fixing how the checker reads citation brackets moved gpt-oss-120b from 40% to **60%** correct and cut first-token p95 from **12,862 ms to 3,082 ms**. The wrong refusals had each triggered a redraft that waited for token budget.

Quality: on my 25 questions (20 answerable, 5 not), the final configuration got 15 correct, 0 wrong, all 5 unanswerable refused, and 5 answerable refused. Across all 175 real graded answers, 3 were wrong, and all three passed the checker. In each, the number *was* in the cited passage but was attached to the wrong thing: "Fisher … 170 hours" (the 170 h belongs to a different dataset), and two h6 answers that described the Chinese-model training set. The number check catches made-up numbers, not numbers moved to the wrong noun.

What the 5 wrong refusals cost:

- m3 and m7: the understand step doesn't know Helium is Moshi's LLM.
- h4: the model computed "6.4 points", a number the paper doesn't print.
- h6: retrieval missed the passage.
- x1: the answer is in a table that PDF extraction scrambles.

Tokens per answer: 1,854–2,668 depending on the setting.

4 Where the time goes

Request `steady-real-0003` (real; "…Moshi's theoretical latency and its latency in practice?") took **13,583 ms to its first token**, and all of the delay came *before* the first token. The traces give each call's timing:

Part	Time	What happened
Working	798 ms	Understand call
Retrying	697 ms	First draft, rejected as an "uncited sentence" (it was the 【n】 bug)
Waiting	11,564 ms	The governor held the redraft until the 120b TPM bucket

Waiting	11,564 ms	The governor held the redraft until the 120b TPM bucket refilled (`wait_ms` in the row)
Working	521 ms	Second draft, rejected again; the system then refused

The total is about 13.6 s. That's how I found that a checker bug turns into latency.

5 When a limit is hit

Per-minute limit

- **Delayed:** requests already in progress keep their place. A redraft goes ahead of new arrivals.
- **Dropped:** a new arrival whose predicted wait is more than its 20 s budget is told within milliseconds.
- **Weaker answer:** if the 120b wait is more than 10 s, the request skips understand and drafts on 20b with 2 passages. That model has its own 8K TPM bucket.
- **Limits that change without notice:** the governor adopts the new limit from Groq's `x-ratelimit-limit-tokens` header and from the "Limit N" in 429 text. With limits halved, give-ups were **31.0%** when the governor ignored the change and **6.9%** when it adopted it.

Per-day limit

- The 120b bucket empties first, so new answers move to 20b.
- When both are empty, the person gets §7's reply in about 1 ms, and cached answers keep flowing.
- I drop people instead of queueing them for minutes, because the brief says people give up sooner than expected.

Retrieval: quality versus cost

All runs were real, with 25 questions each, on gpt-oss-20b with the v1 checker unless noted (Table 3).

Setting	Correct
120 words × 8, re-rank	52%
250 × 4, re-rank	60%
250 × 4, no re-rank	64%
500 × 2, no re-rank	60% (fewest tokens: 1,854)

Re-rank vs no re-rank is one question apart, within noise. Small pieces scored lowest; I didn't check why.

My choice is 250 × 4 without re-rank. It scored best and is simpler. The 120b runs used re-rank, and I'd switch it off.

Near the limit: the system drops to 250 × 2, about 720 context tokens instead of about 1,500. Token-free recall stays at 90%. I did not measure answer quality at that setting.

How this library will mislead a naive retriever

1. **The papers contradict themselves, and the abstract ranks first.**
   - AIAYN's abstract says EN-FR BLEU **41.8**, but §6.1 says **41.0**.
   - H-Net says DNA data efficiency is "nearly **4×**" in its abstract and **3.6×** in its intro.

   "What BLEU did the Transformer get on English-French?" returns the abstract.
2. **Shared vocabulary.** "Heads", "Transformer", "tokens" and "chunk" appear in all three papers. "How many heads does Moshi's Temporal Transformer use?" pulls AIAYN's h = 8 and Mimi's 8 heads. The answer, 32, sits in a table that pdftotext scrambles. Both models refused it, correctly given what they were shown.
3. **Different wording.** "Which dataset were the H-Net LMs trained on?" misses "100B token subset sampled from FineWeb-Edu" in every setting. Conversely, "What BLEU does Moshi get on WMT14?" gets the *highest* retrieval score of all 25 questions (51.7), so the refusal has to come from the model and checker. Retrieval scores alone won't produce it.

6 Retries

Policy:

- At most 2 retries per call.
- Honour `retry-after`, and back off 0.5 s, then 1 s, with jitter, otherwise.
- Retry only if the wait still fits the deadline.
- Never retry a call that already streamed tokens.

Where retrying made things worse (stand-in, limits halved and unknown to the governor): naive retries (3 tries, 0.2 s apart, ignoring `retry-after`) pushed p95 first token from 3,819 ms to **20,181 ms**. In `halved-static-naive-0018`, the draft got a 429, the immediate retry re-queued behind everyone and waited 18.2 s, and the person gave up at 19.8 s.

Where retrying helped (real): `steady-real-0007` hit a dropped connection, retried, and answered in 2,000 ms.

In normal overload, naive retries barely fire (0.02 per request), because the governor prevents 429s. Retries matter exactly when the governor's picture of the limits is wrong.

7 What the person sees

At 3pm, with the daily budget gone, the reply arrives in about 1 ms:

> *"Today's model budget is used up, so I can't write a new checked answer right now. Capacity comes back gradually; try again in about N min. Nobody has asked this exact question today, so there is no saved answer to show you. Meanwhile, these are the passages most likely to answer it. They are raw excerpts, not an answer, and I have not checked them against your question: [paper, page: excerpt] ×3."*

N comes from the measured refill rate (one request per 86 s). It doesn't say "tomorrow", because Groq's daily budget refills continuously. A question already answered today gets that verified answer.

8 What I'm unsure about, what I left out, what I learned

Least certain assumption: people's patience, lognormal with a 12 s median. Every shed threshold comes from it.

- If people wait longer, I'm turning away people I could have served.
- If they give up sooner, the 20 s budget is too long. In stand-in overload, 11 people gave up while still queued.

Deliberately not built:

- An LLM judge, which would double tokens under an 8K TPM ceiling.
- Embeddings or re-ranking models. Three papers and lexical questions didn't justify the extra calls.

Rubric change, disclosed: I widened question m6 after reading answers. The paper fine-tunes on Fisher (2,000 h) *and* on a 170 h "supervised multi-stream dataset", so my question was ambiguous. Saying Fisher has 170 h is still graded wrong. All regrades are in `regrade_log.csv`.

New to me: I had assumed "per-minute" and "per-day" limits reset on the clock, at the top of the minute or at midnight. Groq's don't. They work more like a bucket that refills steadily: 8,000 tokens a minute is really about 133 tokens every second, and 1,000 requests a day means one request comes back every 86 seconds. I saw this in the response headers. The daily reset timer went up by 86.4 s with each request, and after I hit the minute limit I was let back in after 2.7 s, not 60. That changed my design. There is no reset moment to wait for, and a message telling someone to "come back tomorrow" would simply be wrong.

I changed three things: the wording is first-person and conversational, the "bucket" idea is introduced before the numbers, and the closing line now

says why it matters. If I need it shorter for the 2-page limit, I'll cut the 133 tokens/second example and keep the 86.4 s and 2.7 s observations, since those are my evidence.

Your feedback on Vaani

What stood out: The "zero dead air" and turn-taking ideas show the team cares about how a call feels, not just what the agent says. The ~450 ms p90 latency claim and the customer logos back that up.

What confused me: The headline calls Vaani "the only voice AI harness you'll ever need", and I wasn't sure what "harness" meant at first. The start buttons also lead to different places (app.vaanivoice.ai and platform.vaani.dev), so I wasn't sure which was the real starting point.

One thing I'd change: Add one plain line under the headline saying what Vaani does and who it's for, next to a single "Start free" button. That would fix both the jargon and the two entry points.

How I worked with the assistant

I built this with an AI assistant (Claude, in Cowork). My prompts are attached word for word; only my Groq API keys are redacted, and those keys are deleted.

What the assistant produced: nearly all the design and code: the five-step pipeline, the rate governor, the per-sentence verifier, the stand-in, the load generator and the analysis. It also wrote the 25 test questions from the paper text, and the first draft of this report.

What I did: I chose Python. When its sandbox couldn't reach arXiv or Groq, I downloaded and uploaded the three pinned PDFs. I created the Groq account, set up Python on my Windows laptop, ran every real-service measurement and evaluation locally, and connected my folder so every change landed there. I wrote the "New to me" and Vaani sections myself.

Three prompts that mattered:

1. The brief plus "do this assignment".
2. Uploading the three PDFs, which unblocked retrieval.
3. Pasting my first run's output, where every request returned 404.

What it got wrong that I caught: it built everything on Llama models taken from third-party docs. My first real run returned 404 for all of them; I pasted the output back and it switched to gpt-oss-120b/20b. It also first blamed the H-Net refusals on missing citations. Its own logged drafts later showed the real cause was gpt-oss-120b citing as 【3】; that catch was the assistant's, from my run's data, not mine.

Where I overruled it: it recommended leaving the editing note out of my "New to me" section because it read like an editor's comment. I kept it in, and accepted only a first-person wording change. That is a small decision; I did not overrule it on system design.

Distinctly mine: every real-service number comes from my account and my laptop, including the overnight network drop-outs, which are labelled and excluded rather than hidden.

Results tables

Table 1: Limits found in the real service

Limit	Value found	How I found it	Confidence
Requests per minute	30 per model	40 calls at once: 31 ok, 9 × 429 “on requests per minute (RPM): Limit 30, Used 30”	high
Tokens per minute	8,000 per model	`x-ratelimit-limit-tokens` on every response; refills about 133/s (reset grew 5.2 s per 697 tokens)	high
Requests per day	1,000 per model	`x-ratelimit-limit-requests`; reset header grows 86.4 s per call (rolling)	high
Tokens per day	200,000 per model	Groq rate-limit docs, plus a real 429 with retry-after 696 s, consistent with a 200K/day refill (that 429’s body was not logged)	medium
Requests in flight at once	no limit seen at 12	12 concurrent calls to 120b, all 200	low
What it returns when a limit is hit	HTTP 429, JSON `error.type = requests/tokens`, `code = rate_limit_exceeded`, message naming the limit and “try again in Xs”; headers `retry-after` and `x-ratelimit-*`	captured in `probe_raw.jsonl`	high
Time until it recovers	RPM: 2.7 s; daily tokens: 696 s for one ~1.6K-token call	polling every 2 s after the 429; retry-after on the TPD 429	medium

Table 2a: Latency per load scenario

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady-real	real	11	299	2.3	1944 / 13583 / 13583	1944 / 13583 / 13583
steady	standin	27	234	7.6	730 / 7591 / 9517	744 / 9534 / 20836
burst	standin	66	214	19.7	457 / 10300 / 12628	487 / 10313 / 12645
daily-small40left	standin	15	119	8.2	506 / 8797 / 8797	536 / 8846 / 8846
overload	standin	145	317	29.4	357 / 11690 / 14390	408 / 11713 / 14397
halved-static	standin	29	256	7.4	632 / 3819 / 11566	652 / 3826 / 11585
burst-nocache	standin	62	249	16.0	1001 / 9991 / 15049	1016 / 10027 / 15060
overload-naive	standin	171	315	34.6	0 / 10298 / 12937	0 / 10311 / 12957
halved	standin	29	256	7.4	688 / 7580 / 10045	709 / 7600 / 10058
daily	standin	15	119	8.2	622 / 8837 / 8837	644 / 8887 / 8887
halved-static-naive	standin	19	215	5.9	697 / 20181 / 20181	704 / 20219 / 20219

Table 2b: Outcomes, same rows

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
steady-real	100.0	0.0	0.0	0.36	1	2424	$0.43
steady	96.3	3.7	0.0	0.15	1	1553	$0.28
burst	59.1	34.8	6.1	0.03	3	933	$0.16
daily-small40left	93.3	6.7	0.0	0.13	1	1035	$0.12
overload	79.3	13.1	7.6	0.01	3	788	$0.11
halved-static	69.0	0.0	31.0	0.34	2	1147	$0.17
burst-nocache	38.7	50.0	11.3	0.06	3	1664	$0.26
overload-naive	71.3	12.3	16.4	0.02	3	677	$0.10
halved	79.3	13.8	6.9	0.14	2	1201	$0.17
daily	46.7	53.3	0.0	0.13	1	969	$0.13
halved-static-naive	73.7	5.3	21.1	0.53	1	1553	$0.24

Table 3: Retrieval settings (real, 25 questions each)

Setting (piece size, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
quality-small-k8-rr-sentence-20b (real)	25	52.0	4.0	20.0	2107	1150 / 1912	1150 / 2144
quality-mid-k4-rr-sentence-20b (real)	25	60.0	0.0	20.0	2198	1342 / 3174	1342 / 3174
quality-mid-k4-norr-sentence-20b (real)	25	64.0	4.0	20.0	2041	1467 / 3343	1487 / 3343
quality-mid-k4-rr-sentence-120b (real)	25	40.0	0.0	20.0	2668	1651 / 12862	1707 / 12862
quality-large-k2-norr-sentence-20b (real)	25	60.0	4.0	20.0	1854	1181 / 7156	1183 / 7156
quality-mid-k4-rr-group-120b (real)	25	44.0	0.0	20.0	2355	1545 / 12786	1616 / 12786
quality-mid-k4-rr-group-120b_citefix (real)	25	60.0	0.0	20.0	1997	1335 / 3082	1335 / 3158

Refused wrongly % per setting:

- quality-large-k2-norr-sentence-20b: 16.0% (n=25), errors/non-ok: 0
- quality-mid-k4-norr-sentence-20b: 12.0% (n=25), errors/non-ok: 0
- quality-mid-k4-rr-group-120b: 36.0% (n=25), errors/non-ok: 0

- quality-mid-k4-rr-group-120b_citefix: 20.0% (n=25), errors/non-ok: 0
- quality-mid-k4-rr-sentence-120b: 40.0% (n=25), errors/non-ok: 0
- quality-mid-k4-rr-sentence-20b: 20.0% (n=25), errors/non-ok: 0
- quality-small-k8-rr-sentence-20b: 24.0% (n=25), errors/non-ok: 0

Excluded from tables (laptop lost network; HTTP status 0, request never reached Groq): quality-large-k2-norr-20b-netfail (25 rows: {'error': 20, 'ok': 4, 'rate_limited': 1}), quality-mid-k4-rr-120b-netfail (8 rows: {'error': 8})

Cost uses assumed Groq paid prices of $0.15/$0.60 (120b) and $0.075/$0.30 (20b) per million input/output tokens. I have not checked these; on the free tier the actual cost is $0. Peak queue length comes from the governor's own counter (`results/peaks.jsonl`), because the CSV cannot reconstruct it. The excluded rows are listed under the tables and stay in `results.csv`.